

Flutter & Dart Viva — Clean Point-Wise Answers

Simple, viva-friendly answers to all 25 questions — short enough to remember, complete enough to explain if a follow-up is asked.

SET-1

1. What is Flutter?

- **Flutter** is an open-source UI framework developed by **Google**.
 - It is used to build applications for **Android, iOS, Web, Windows, macOS, and Linux**.
 - Flutter uses the **Dart programming language**.
 - It allows developers to create applications from a **single codebase**.
 - Flutter uses **widgets** to build the user interface.
- Viva answer:** > Flutter is Google’s open-source UI framework used to build cross-platform applications using Dart from a single codebase.

2. What is the purpose of the build() method in Flutter?

- The `build()` method describes **what the UI of a widget should look like**.
- It returns a **Widget**.
- It receives a `BuildContext` parameter.
- Flutter calls `build()` when the widget needs to be **rendered or rebuilt**.

```
Widget build(BuildContext context) {  
  return Text("Hello");  
}
```

Viva answer: > The `build()` method returns the widget tree that describes the UI of a widget.

3. What is the purpose of Row, Column and Stack widgets?

Row - Arranges widgets **horizontally**. - Main axis → horizontal.

```
Row(  
  children: [  
    Text("A"),  
    Text("B"),  
  ],  
)
```

Column - Arranges widgets **vertically**. - Main axis → vertical.

```
Column(  
  children: [  
    Text("A"),  
    Text("B"),  
  ],  
)
```

Stack - Places widgets **on top of each other**. - Useful for overlapping elements.

Easy memory: > **Row = side by side · Column = top to bottom · Stack = one over another**

4. What is the difference between final and const ?

final - Value can be assigned **only once**. - The value can be determined **at runtime**.

```
final name = getName();
```

const - Value must be known at **compile time**. - It creates a compile-time constant.

```
const pi = 3.14;
```

Viva answer: > `final` means the variable can be assigned only once, while `const` means the value is a compile-time constant.

5. How does runApp() differ from main() ?

main() - It is the **entry point of a Dart program**. - Program execution starts from `main()`.

```
void main() {  
  runApp(MyApp());  
}
```

runApp() - It is a Flutter function. - It takes the **root widget** and displays it on the screen.

Remember: > `main()` → starts the Dart program · `runApp()` → starts the Flutter UI

SET-2

1. Difference between Stateful and Stateless widgets?

StatelessWidget - Does **not have mutable state**. - Its UI depends on the values given to it. - It does not change its internal state during its lifetime.

```
class MyWidget extends StatelessWidget {
```

StatefulWidget - Can have **changing state**. - Uses a separate `State` object. - `setState()` is commonly used to update the UI.

```
class MyWidget extends StatefulWidget {
```

Easy memory: > **Stateless = UI doesn’t maintain changing internal state · Stateful = UI can change because its state changes**

2. What is a MaterialApp in Flutter?

- `MaterialApp` is a widget that provides **Material Design** features.
- It is commonly used as the **root widget** of a Flutter application.
- It provides things such as: Theme, Navigation, Routes, App title, Material design styling.

```
MaterialApp(  
  home: HomePage(),  
);
```

Viva answer: > `MaterialApp` provides the basic Material Design structure and configuration for a Flutter application.

3. What are mainAxisAlignment and crossAxisAlignment ?

Properties used mainly with **Row** and **Column**.

mainAxisAlignment — Controls alignment along the **main axis** (Row → horizontal, Column → vertical).

```
mainAxisAlignment: MainAxisAlignment.center
```

crossAxisAlignment — Controls alignment along the **cross axis** (Row → vertical, Column → horizontal).

Easy trick: > **Main = direction the Row/Column is going · Cross = opposite direction**

4. What is meant by Null-aware operators?

Dart operators used to **safely work with null values**.

- `?.` — Access something only if it isn’t null: `name?.length`
- `??` — Provides a default value if something is null: `String name = value ?? "Unknown";`
- `??=` — Assigns a value only if the variable is null: `name ??= "Tim";`

Viva answer: > Null-aware operators help us safely handle null values without causing null-related errors.

5. What is Dart?

- **Dart** is a programming language developed by **Google**.
 - Flutter applications are primarily written in Dart.
 - It supports: object-oriented programming, classes and objects, null safety, asynchronous programming, strong typing.
- Viva answer:** > Dart is Google’s programming language used to develop Flutter applications.

SET-3

1. What are the advantages and limitations of Flutter?

Advantages

1. **Cross-platform** — one codebase can target multiple platforms.
2. **Fast development** — supports Hot Reload.
3. **Rich widgets** — provides many ready-made UI widgets.
4. **Good performance** — Flutter uses its own rendering engine.
5. **Customizable UI** — interfaces can be highly customized.
6. **Single codebase** — reduces development effort.

Limitations

1. Application size can be relatively large.
2. Some platform-specific features may require native code/plugins.
3. Developers need to learn **Dart**.
4. Third-party packages may vary in quality or support.
5. Platform-specific behavior can sometimes require additional work.

2. What is a Widget? What are the different types of widgets in Flutter?

Widget - A widget is a **building block of the Flutter UI**. - Everything in Flutter's UI is represented using widgets. - Widgets describe how the UI should look and behave.

Examples: `Text`, `Image`, `Container`, `Row`, `Column`, `Scaffold`

Main types

1. **StatelessWidget** — Does not maintain changing internal state.
2. **StatefulWidget** — Maintains state that can change during runtime.

Viva answer: > A widget is a basic building block of a Flutter user interface. The two fundamental types are `StatelessWidget` and `StatefulWidget`.

3. What is Scaffold in Flutter?

- `Scaffold` provides the **basic visual structure** of a Material Design screen.
- It provides common areas such as: `AppBar`, `body`, `drawer`, `bottomNavigationBar`, `floatingActionButton`.

```
Scaffold(  
  appBar: AppBar(  
    title: Text("Home"),  
  ),  
  body: Text("Hello"),  
);
```

Viva answer: > Scaffold provides the basic structure of a Material Design screen in Flutter.

4. What is a constructor? Name types of constructors in Dart.

Constructor - A constructor is a special method used to **create and initialize an object**. - It has the **same name as the class**.

```
class Student {  
  Student() {  
    print("Created");  
  }  
}
```

Types of constructors

1. Generative/default constructor
2. Named constructor
3. Parameterized constructor
4. Constant (`const`) constructor
5. Factory constructor
6. Redirecting constructor

```
class Student {  
  Student();  
  Student.named();  
}
```

Viva answer: > A constructor initializes an object when it is created. Dart supports generative, named, `const`, factory, and redirecting constructors.

5. What is pubspec.yaml file?

- `pubspec.yaml` is an important configuration file in a Flutter/Dart project.
- It contains: project name, version, dependencies, Flutter SDK configuration, assets, fonts.

```
dependencies:  
  flutter:  
    sdk: flutter  
  http: ^1.0.0
```

Viva answer: > `pubspec.yaml` is the configuration file that defines project metadata, dependencies, assets, fonts, and Flutter settings.

SET-4

1. What do “Hot Reload” and “Hot Restart” mean in Flutter?

Hot Reload - Applies code changes to the running application. - Usually **preserves the current state**. - Very fast; useful while designing and debugging UI.

Hot Restart - Restarts the Flutter application. - **Resets the application state**. - Takes longer than hot reload.

Easy memory: > **Hot Reload** → **change UI, keep state** · **Hot Restart** → **restart app, lose state**

2. What is a Container widget and what are its important properties?

- `Container` is a widget used to **contain and decorate another widget**.
- It can control: size, padding, margin, alignment, background color, border, border radius, decoration.

```
Container(  
  width: 100,  
  height: 100,  
  padding: EdgeInsets.all(10),  
  color: Colors.blue,  
  child: Text("Hello"),  
)
```

Important properties: `width`, `height`, `padding`, `margin`, `alignment`, `color`, `decoration`, `child`

3. List the keywords in Dart programming.

<code>var</code>	<code>final</code>	<code>const</code>	<code>late</code>	<code>dynamic</code>
<code>class</code>	<code>extends</code>	<code>implements</code>	<code>with</code>	<code>new</code>
<code>this</code>	<code>super</code>	<code>static</code>	<code>abstract</code>	<code>interface</code>
<code>enum</code>	<code>if</code>	<code>else</code>	<code>for</code>	<code>while</code>
<code>do</code>	<code>switch</code>	<code>case</code>	<code>break</code>	<code>continue</code>
<code>return</code>	<code>try</code>	<code>catch</code>	<code>finally</code>	<code>throw</code>
<code>assert</code>	<code>import</code>	<code>export</code>	<code>library</code>	<code>part</code>
<code>void</code>	<code>async</code>	<code>await</code>	<code>yield</code>	

For a viva, you do **not** need to explain every keyword unless asked.

4. Difference between a List, Set and Map?

List — Ordered collection, allows duplicate values, uses an index.

```
List<int> nums = [1, 2, 2, 3];
```

Set — Collection of unique values, no duplicates.

```
Set<int> nums = {1, 2, 3};
```

Map — Stores key-value pairs; keys are used to access values.

```
Map<String, int> marks = {  
  "Math": 90,  
  "English": 85  
};
```

Easy memory: > **List** → **ordered values** · **Set** → **unique values** · **Map** → **key : value**

5. Difference between a Package and a Plugin?

Package - A reusable piece of Dart/Flutter code. - Usually provides functionality that can be implemented in Dart. - Example: a package for utilities or state management.

Plugin - A package that provides access to **platform-specific functionality**. - It may contain native Android/iOS/etc. code. - Examples: accessing camera, GPS, Bluetooth, etc.

Viva answer: > A package provides reusable Dart/Flutter functionality, while a plugin can also interact with platform-specific native APIs.

SET-5

1. What is the purpose of the BuildContext parameter in build() ?

- `BuildContext` represents the **location of a widget in the widget tree**.
- It allows a widget to access information about its position and surrounding widgets.

Commonly used to access: Theme, Navigator, MediaQuery, MediaQuery, and other shared information.

```
Widget build(BuildContext context) {  
  return Text(  
    "Hello",  
    style: Theme.of(context).textTheme.bodyLarge,  
  );  
}
```

Viva answer: > BuildContext represents a widget's location in the widget tree and allows it to access information and services associated with that location.

2. What is a layout widget in Flutter?

- A layout widget controls **how other widgets are arranged on the screen**.
- It controls position, size, spacing, and alignment.

Examples: Row, Column, Stack, Container, Expanded, Padding, Center, SizedBox

Viva answer: > A layout widget controls the size, position, alignment, and arrangement of widgets in the UI.

3. How do you add images in Flutter?

A. Asset image 1. Put the image inside the project, e.g. assets/images/photo.png 2. Declare it in pubspec.yaml:

```
flutter:  
  assets:  
    - assets/images/photo.png
```

3. Use:

```
Image.asset("assets/images/photo.png")
```

B. Network image

```
Image.network("https://example.com/image.png")
```

Viva answer: > Local images are added as assets and displayed using Image.asset(), while online images can be displayed using Image.network().

4. What are the basic data types in Dart?

Type	Description	Example
int	Whole numbers	int age = 20;
double	Decimal numbers	double price = 10.5;
num	Can be int or double	num value = 10.5;
String	Text	String name = "Tim";
bool	True/false	bool isActive = true;
List	Ordered collection	List<int> numbers = [1, 2, 3];
Set	Unique collection	Set<int> numbers = {1, 2, 3};
Map	Key-value collection	Map<String, int> marks = {"Math": 90};
dynamic	Can hold different types	dynamic value = 10; value = "Hello";

Most important primitive types to remember: > int, double, num, String, bool, and dynamic.

5. What are positional vs named parameters in Dart?

Positional parameters — Arguments are passed according to their **position/order**, written without {}.

```
void greet(String name, int age) {  
  print(name);  
  print(age);  
}  
  
greet("Tim", 20);
```

Named parameters — Arguments are passed using their **parameter names**, written inside {}. Order does not matter.

```
void greet({required String name, required int age}) {  
  print(name);  
  print(age);  
}  
  
greet(name: "Tim", age: 20);  
greet(age: 20, name: "Tim");
```

Easy memory: > Positional → order matters · Named → name matters

Super-Quick Revision Sheet

If you have 5 minutes before the viva, memorize these:

Question	One-line answer
Flutter	Google's cross-platform UI framework using Dart
build()	Returns the widget tree/UI
Row	Arranges widgets horizontally
Column	Arranges widgets vertically
Stack	Overlaps widgets
final	Assigned only once; runtime value allowed
const	Compile-time constant
main()	Entry point of Dart program
runApp()	Starts the Flutter UI
Stateless	Doesn't maintain changing internal state
Stateful	Maintains mutable state
MaterialApp	Provides Material Design app structure
Main axis	Direction of Row/Column
Cross axis	Direction perpendicular to main axis
Null-aware	Safely handles null values
Dart	Programming language used by Flutter
Scaffold	Basic structure of a Material screen
Constructor	Initializes an object
pubspec.yaml	Project configuration/dependencies/assets
Hot Reload	Changes code while generally preserving state
Hot Restart	Restarts app and resets state
Container	Widget for sizing, spacing and decoration
List	Ordered, duplicates allowed
Set	Unique values

Map	Key-value pairs
Package	Reusable Dart/Flutter functionality
Plugin	Package that can access native platform features
BuildContext	Widget's location in widget tree
Layout widget	Arranges/positions widgets
<code>Image.asset()</code>	Displays local asset image
<code>Image.network()</code>	Displays network image
Positional parameter	Order matters
Named parameter	Parameter name matters

This covers all five sets (25 questions).